CSC 3150 Assignment1 Report

Task1 Design: The core design of Program 1 is to use parent–child process cooperation to monitor the execution state of a test program. The program first creates a child process using fork(), and the child process calls execvp() to execute the user-specified test program, effectively replacing its own image. The parent process uses waitpid() with WUNTRACED and WCONTINUED flags to continuously monitor state changes of the child. By utilizing macros such as WIFEXITED, WIFSIGNALED, WIFSTOPPED, and WIFCONTINUED, the parent identifies different termination conditions and prints corresponding messages. A helper function sig_to_name() converts signal numbers into readable names for clarity.

Task2 Design: Program 2 is implemented as a kernel module whose core design is to create a kernel thread (kthread) that, in kernel space, spawns a child process to execute a user program and then waits for and decodes the child's exit status. On module init, kthread_run() starts the my_fork thread; inside the thread the current task's signal actions are reset to defaults to ensure expected signal behavior for the child. The thread uses kernel_clone() to create a child; the child runs my_exec, which invokes kernel_execve() (with getname_kernel/putname) to execute /tmp/test. The parent uses kernel_wait() to wait for the child to terminate, then inspects the status bitfields to determine normal exit, stop-by-signal, or termination-by-signal, and maps signal numbers to human-readable names via map_status(); events and codes are emitted via printk() to the kernel log. The design performs process creation, execution, and status monitoring entirely in kernel context, demonstrating in-kernel process control and exit-status interpretation.

Bonus Design: This program is a /proc-based process-tree utility that prints the system's process hierarchy in a readable tree format. The design first scans the /proc directory for numeric entries and constructs an in-memory Proc record for each process, holding pid, ppid, name, a dynamic children vector, and an alive flag. The

display name is primarily read from /proc/<pid>/comm (read_comm), and the parent PID is extracted by parsing /proc/<pid>/stat (read_ppid_and_name_from_stat), carefully handling process names enclosed in parentheses (which may contain spaces or parentheses). After gathering all entries, processes are linked into their parent's children list by ppid, and each child list is sorted using a comparator that orders by process name first and pid second (cmp_by_name_then_pid). For output, the program starts at PID 1 (init/systemd) and recursively prints the tree using branch glyphs (├─, └─, │ ,) to show hierarchy. It also inspects /proc/<pid>/task to count threads other than the main thread and annotates thread counts as N*[{name}]. Utility helpers (memory wrappers xmalloc/xrealloc/die and a dynamic IntVec with iv_push) simplify allocation and error handling. main() simply calls load_proc() to build the table and print_tree() to emit the formatted tree.

Environment Setup & Compiling Kernel:

Based on an x86-64 processor and Windows 11 system, first download VirtualBox and the corresponding Linux image, then load it.
After that, configure the network settings by selecting NAT mode, and set the port forwarding as Host 2200 → Guest 22.

Once the virtual machine is running, execute the following commands inside the VM:

sudo dhclient / ip a / ssh -p 2200 csc3150@127.0.0.1

This allows remote connection to the Linux virtual machine through Visual Studio Code.

Next, you need to expand the disk space.
Refer to the tutorial document for detailed steps and extend the disk size to ≥50GB.
Also, make sure to allocate at least 4GB of memory during runtime; otherwise, the system may fail to compile or run the kernel properly.

Under the Linux 5.15.10 system, the kernel can be compiled as follows:

Download the kernel source (Tsinghua mirror):

wget https://mirror.tuna.tsinghua.edu.cn/kernel/v5.x/linux-5.15.10.tar.xz

Install required dependencies: apt-get update

apt-get install -y libncurses-dev gawk flex bison openssl libssl-dev dkms \

libelf-dev libudev-dev libpci-dev libiberty-dev autoconf llvm dwarves bc

Extract and move the source code: tar xvf linux-5.15.10.tar.xz

mkdir -p /home/seed/work mv linux-5.15.10 /home/seed/work/

cd /home/seed/work/linux-5.15.10

Configure and compile the kernel:

make mrproper make clean make menuconfig make -j$(nproc)

Install and verify the kernel: make modules_install make install

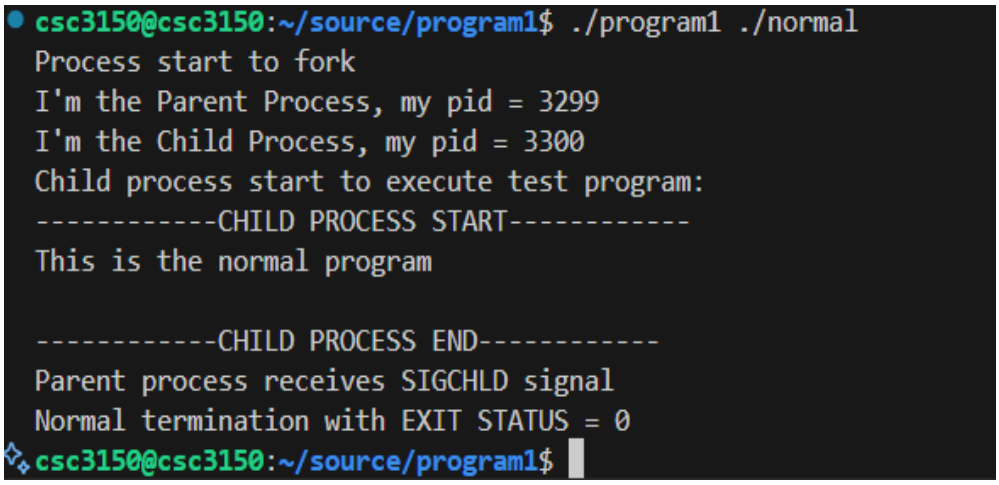
Reboot uname -r

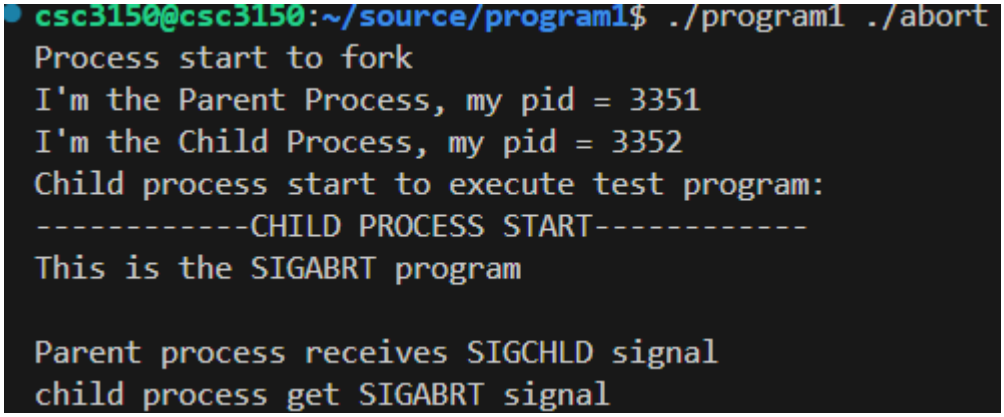
Demo Output for Task1 (Usages as follows)

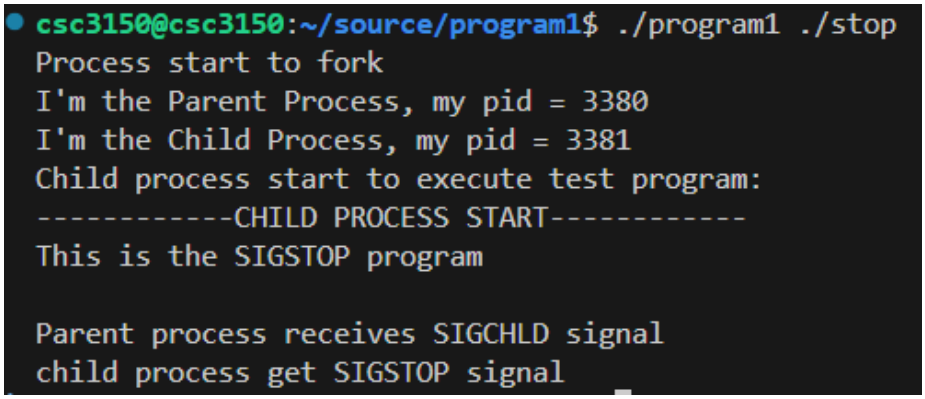
make clean

make

./program1 ./normal







Demo Output for Task2 (Usages as follows)

make clean

make

gcc -o test test.c

cp test /tmp/test

sudo insmod program2.ko

sudo rmmod program2

dmesg

TIPS

DO NOT FORGET TO EXPORT FUNCTION

fs/namei.c ->

EXPORT_SYMBOL(getname_kernel); EXPORT_SYMBOL(putname);

kernel/fork.c –

EXPORT_SYMBOL(kernel_clone);

fs/exec.c->

EXPORT_SYMBOL(kernel_execve);

kernel/exit.c-> EXPORT_SYMBOL(kernel_wait);

Then recompile the kernel:

make menuconfig -> make -j$(nproc) -> make modules_install ->make install -> reboot

```
csc3150@csc3150:~/source/program2$ make clean
make -C /lib/modules/5.15.10/build M=/home/csc3150/source/program2 clean
make[1]: Entering directory '/home/seed/work/linux-5.15.10'
make[1]: Leaving directory '/home/seed/work/linux-5.15.10'
csc3150@csc3150:~/source/program2$ make
make -C /lib/modules/5.15.10/build M=/home/csc3150/source/program2 modules
make[1]: Entering directory '/home/seed/work/linux-5.15.10'
  CC [M]  /home/csc3150/source/program2/program2.o
/home/csc3150/source/program2/program2.c: In function 'my_exec':
/home/csc3150/source/program2/program2.c:80:9: warning: ISO C90 forbids mixed declarations and code [-Wdeclaration-after-statement]
   80 |         long rc = kernel_execve(fn->name, argv, envp);
      |         ^~~~
  MODPOST /home/csc3150/source/program2/Module.symvers
  CC [M]  /home/csc3150/source/program2/program2.mod.o
  LD [M]  /home/csc3150/source/program2/program2.ko
  BTF [M] /home/csc3150/source/program2/program2.ko
make[1]: Leaving directory '/home/seed/work/linux-5.15.10'
```

```
csc3150@csc3150:~/source/program2$ gcc -O2 -o test test.c
csc3150@csc3150:~/source/program2$ sudo cp test /tmp/test
[sudo] password for csc3150:
csc3150@csc3150:~/source/program2$ sudo insmod program2.ko
csc3150@csc3150:~/source/program2$ sudo rmmod program2
csc3150@csc3150:~/source/program2$ dmesg
[    0.000000] Linux version 5.15.10 (csc3150@csc3150) (gcc (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0, GNU ld (GNU Binutils for Ubuntu) 2.34) #2 SMP Sun Sep 28 12:42:42 UTC 2025
[    0.000000] Command line: BOOT_IMAGE=/vmlinuz-5.15.10 root=/dev/mapper/ubuntu--vg-ubuntu--lv ro auto=true preseed/file=/cdrom/preseed.cfg priority=critical quiet splash noprompt noshell automatic-ubiquity debian-installer/locale=en_US keyboard-configuration/layoutcode=us languagechooser/language-name=English localechooser/supported-locales=en_US.UTF-8 countrychooser/shortlist=CN --
[    0.000000] KERNEL supported cpus:
[    0.000000]   Intel GenuineIntel
[    0.000000]   AMD AuthenticAMD
[    0.000000]   Hygon HygonGenuine
[    0.000000]   Centaur CentaurHauls
[    0.000000]   zhaoxin   Shanghai
[    0.000000] [Firmware Bug]: TSC doesn't count with P0 frequency!
[    0.000000] x86/fpu: x87 FPU will use FXSAVE
[    0.000000] signal: max sigframe size: 1440
[    0.000000] BIOS-provided physical RAM map:
[    0.000000] BIOS-e820: [mem 0x0000000000000000-0x000000000009fbff] usable
[    0.000000] BIOS-e820: [mem 0x000000000009fc00-0x000000000009ffff] reserved
[    0.000000] BIOS-e820: [mem 0x00000000000f0000-0x00000000000fffff] reserved
[    0.000000] BIOS-e820: [mem 0x0000000000100000-0x00000000dffeffff] usable
[    0.000000] BIOS-e820: [mem 0x00000000dfff0000-0x00000000dfffffff] ACPI data
[    0.000000] BIOS-e820: [mem 0x00000000fec00000-0x00000000fec00fff] reserved
[    0.000000] BIOS-e820: [mem 0x00000000fee00000-0x00000000fee00fff] reserved
[    0.000000] BIOS-e820: [mem 0x00000000fffc0000-0x00000000ffffffff] reserved
[    0.000000] BIOS-e820: [mem 0x0000000100000000-0x000000019fffffff] usable
[    0.000000] NX (Execute Disable) protection: active
```

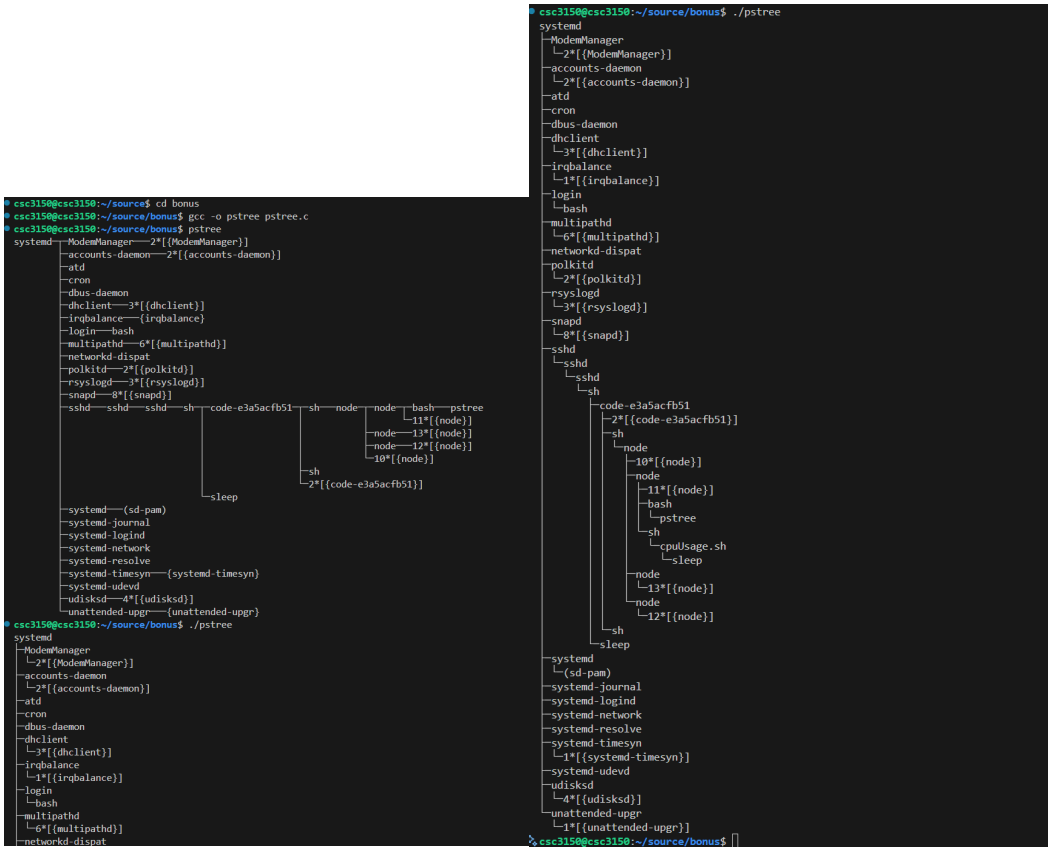
```
[ 1939.834276] [program2] : module_init
[ 1939.835758] [program2] : module_init create kthread start
[ 1939.844437] [program2] : module_init kthread start
[ 1939.844679] [program2] : This is the parent process, pid = 5220
[ 1939.845846] [program2] : The child process has pid = 5222
[ 1939.846024] [program2] : child process
[ 1939.846694] [program2] : get SIGTERM signal
[ 1939.846871] [program2] : child process terminated
[ 1939.847038] [program2] : The return signal is 15
[ 1987.388616] [program2] : module_exit
[ 2070.842998] [program2] : module_init
[ 2070.843276] [program2] : module_init create kthread start
[ 2070.845660] [program2] : module_init kthread start
[ 2070.845856] [program2] : This is the parent process, pid = 5756
[ 2070.847883] [program2] : The child process has pid = 5758
[ 2070.848086] [program2] : child process
[ 2070.961195] [program2] : get SIGBUS signal
[ 2070.961464] [program2] : Child produced a core dump
[ 2070.961645] [program2] : child process terminated
[ 2070.961817] [program2] : The return signal is 7
[ 2075.679610] [program2] : module_exit
```

Demo Output for Bonus(I just completed the basic function of pstree, I did not implement the options. Usages as follows)

make clean

make

./pstree



Experience from Assignment1

Task1: We developed a user-space program that uses fork() to create child processes, executes test programs, and handles termination using wait() and signals (SIGCHLD). This task helped me understand the lifecycle of a Linux process, including:

1. Process creation and hierarchy: Learned how fork() creates a child process that shares certain resources with the parent while maintaining its own address space.
2. Process termination: Understood how wait() retrieves the child's exit status and how to distinguish between normal (exit()) and abnormal (signals such as SIGSEGV, SIGABRT) terminations.
3. Signal handling: Learned how a parent process captures and interprets signals from a child to determine why it terminated.
4. Makefile usage: Practiced automating compilation with Makefile, reinforcing the importance of portability and structured build processes.

Task2: we modified the program2.c template to implement process creation, execution, and signal handling within a kernel module.
This task provided practical insights into the internal workings of the Linux kernel, including:

1. Kernel threads and module lifecycle: Learned how to use kthread_create, module_init, and module_exit to manage kernel module initialization and cleanup.
2. Kernel-level process creation: Studied how _do_fork() and do_execve() work at the kernel level to create and execute processes.
3. Signal propagation inside the kernel: Understood how the kernel uses do_wait() to track child process states and signals such as SIGTERM.
4. Kernel logging and debugging: Practiced using printk() to log kernel messages and analyzed them with dmesg.

Bonus: In the Bonus Task, we implemented a simplified version of the pstree command by reading from the /proc filesystem to display the process hierarchy. From this exercise, I learned:

1. Structure of the /proc filesystem: Understood how process-related information is stored under /proc/[pid]/ directories (e.g., status, cmdline, stat).
2. System calls and directory traversal: Learned how to iterate through /proc entries in user space and build relationships between parent and child processes.
3. Tree representation of processes: Practiced constructing hierarchical process trees for visualization.